



Universitatea „Ștefan cel Mare” Suceava

Facultatea de Inginerie Electrică și Știința Calculatoarelor

PROIECT DE DIPLOMĂ

Robot bazat pe orientare autonomă și

inteligență artificială

Coordonator științific:

Ș.l. dr. ing. ZAGAN Ionel

Absolvent

POLOCOȘER Rebeca-Mihaela

SUCEAVA

2023

CUPRINS

Rezumat	3
Introducere	4
Capitolul 1. Delimitarea de alte proiecte anterioare realizate	5
1.1. Rol educațional	5
1.2. Divertisment	6
1.3. Căutare și salvare	7
1.4. Dezamorsare de bombe	8
1.5. Robot aspirator.....	9
Capitolul 2. Componente utilizate	10
2.1. Arduino Nano	12
2.2. Senzor HC-SR04.....	15
2.3. Driver	16
2.4. Motor de curent continuu	19
2.5. Sisteme periferice.....	20
Capitolul 3. Considerații de implementare	21
Capitolul 4. Rezultate experimentale	30
Capitolul 5. Concluzii și direcții viitoare de dezvoltare	33
Capitolul 6. Bibliografie	34
Anexe	36

Rezumat

În viața de zi cu zi se întâlnesc tot mai multe ajutoare robotizate ale casei, în armată, în industrie, în învățământ, medicină, etc. Una din categoriile acestor roboți ar fi unul de tip Maze-Runner sau un robot mobil care să știe să se deplaseze într-un spațiu limitat și să delimiteze spațiul într-o hartă în memoria acestuia, pentru a utiliza pe viitor această încăpere mai optim.

Lucrarea de față are ca scop crearea și implementarea unui robot de mici dimensiuni având capacitatea de a se orienta autonom într-un labirint, precum și auto-învățarea traseului deja parcurs.

Inițial, robotul este introdus la începutul unui labirint, iar cu ajutorul a doi senzori ultrasonici el se va deplasa în labirint. Când acesta va ajunge la ieșire, acesta se va opri, dându-și seama că se află într-un spațiu nelimitat. Traseul care a fost parcurs din momentul de start până la momentul final este memorat cu ajutorul unui algoritm de inteligență artificială, creat în totalitate de subsemnatul. După ce este poziționat la început, pentru a doua oară, robotul va parcurge labirintul pe cea mai scurtă cale, adică pe calea memorată la prima parcurgere a traseului.

Ca rezultat, se dorește de a obține un robot mobil stabil din punct de vedere mecanic și electronic, ca ulterior să se poată implementa aplicația elaborată pentru acțiunea de bază a acestuia. În urma licenței realizate se așteaptă observarea punctelor slabe și motivele viitoarelor îmbunătățiri a proiectului. Realizarea unei concluzii la etapa finală a sistemului prezentat.

Introducere

Oamenii se străduiesc în permanență să eficientizeze și să depășească multele provocări întâmpinate pe parcursul anilor. Din această cauză și al nivelului ridicat de tehnologizare, au început să se creeze roboți și sisteme inteligente. Aceștia au ca scop efectuarea unor sarcini repetitive într-un mod cât mai eficient, productiv și în siguranță. În unele domenii, precum producția industrială, explorarea spațială, medicină, asistență personală, securitate și supraveghere, roboții au ajuns să dețină un rol important. Putem spune că ~~am~~ oamenii au ajuns dependenți de ajutorul acestor tehnologizări.

Roboții sunt operatori mecanici sau artificiali compuși din baterii și surse de alimentare, senzori, unitate de procesare, software, având o structură care poate varia în funcție de aplicație și specificațiile dorite. Cu ajutorul tuturor acestor componente, un robot inteligent poate avea capacitatea de a percepe mediul înconjurător și de a interacționa cu el sau chiar de a îmbunătăți comportamentul în funcție de experiența obținută în urma efectuării anumitor task-uri repetitive sau în urma analizei amănunțite a mediului de lucru.

Importanța acestor tipuri specifice de roboți care reușesc să repereze cel mai optim traseu rezidă în potențialul lor de a rezolva probleme complexe. Aceste abilități pot fi puse în aplicare în domeniul spațial, unde roboții sunt utilizați pentru a naviga prin structuri necunoscute, precum și în domeniul căutării și salvării.

Pentru ca roboții inteligenți să rezolve problema de a găsi cel mai scurt drum într-un labirint, se utilizează diferite combinații de algoritmi precum și diferite tehnici avansate.

Bazat pe informațiile de mai sus a fost conceput un robot bazat pe orientare autonomă și inteligență artificială. Acesta este introdus în labirint și parcurge tot traseul până ajunge la ieșire, memorând prin anumiți algoritmi creați specific proiectului acesta, traseul cel mai scurt. Când este introdus a doua oară în acest labirint, el va reuși să găsească ieșirea în timp optim. La baza sistemului stă o placă Arduino Nano, precum și diferiți senzori pentru a putea avea o deplasare cât mai fluidă în labirint.

Capitolul 1. Delimitarea de alte proiecte anterioare realizate

Roboții sunt folosiți în diferite domenii și reușesc să ajute oamenii prin multe metode. În acest capitol vor fi prezentate câteva din aceste domenii care folosesc în special algoritmul de deplasare printr-un labirint, sau diferite cazuri asemănătoare.

1.1. Rol educațional

Pentru a putea asimila mai ușor și eficient informațiile predate de profesori, roboții au fost introduși în diferite instituții școlare. Aceștia sunt folosiți atât la nivelul copiilor de la grădiniță, cât și pentru elevii de la clasele primare și gimnaziale cu rolul de a învăța în mod practic anumite noțiuni de bază.

Spre exemplu, elevii pot construi robotul observând toate componentele din care este realizat. Pot crea un labirint mai complex sau mai puțin complex cu diferite obstacole, pereți sau bariere. Programarea folosind programe accesibile precum Blockly sau Python poate fi de multe ori provocatoare. Pot fi setate și testate mișcările robotului cu mare ușurință. Prin intermediul acestei aplicații, elevii pot asimila abilități de rezolvare a diferitelor tipuri de probleme, deoarece ei trebuie să identifice obstacolele, să găsească soluții și să creeze strategii eficiente pentru a găsi drumul corect prin labirint. De asemenea, învățarea interactivă poate oferi o experiență tangibilă care ajută utilizatorii să-și dezvolte gândirea logică și abilitățile de programare.



Figura 1.1. Robot de învățare [\[1\]](#)

Prin diferite aplicații, roboții sunt utilizați în stimularea interesului pentru domenii complexe precum știința, tehnologia, matematica și ingineria. Astfel se pot dezvolta competențe esențiale care vor fi utilizate mai apoi în societate și la locul de muncă într-un mod captivant și interactiv.

1.2. Divertisment

Roboții care merg prin labirint pot fi implicați în diferite activități de divertisment, oferind o experiență interactivă pentru utilizatori și nu numai. În zilele noastre, tinerii din toată lumea sunt provocați de a crea un sistem inteligent cu scopul de a parcurge în cel mai scurt timp anumite trasee.

Un exemplu de acest fel de concurs a apărut în 1970. Scopul final al roboților care sunt sub numele de micro-șoarece este de a rezolva un labirint de dimensiunea 16x16 într-un timp cât mai scurt. Regulamentul acestui concurs spune că robotul trebuie să se poată deplasa în dimensiunile oficiale ale labirintului, nu trebuie să zboare, să zgârie pereții, nu trebuie să marcheze, să deterioreze sau să distrugă sub orice formă pereții labirintului. Lungimea și lățimea robotului nu trebuie să depășească dimensiunea de 25cm.

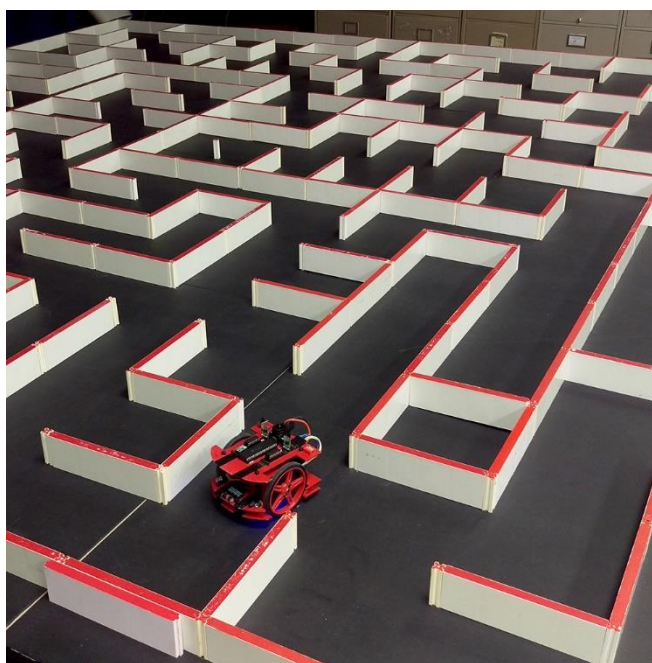


Figura 1.2. Labirint pentru concursurile micro-șoarece [\[2\]](#)



Figura 1.3. Cel mai rapid micro-șoarece [\[3\]](#)

Evenimentul a ajuns popular în Japonia, India, Coreea de Sud, Singapore, Regatul Unit, precum și în Statele Unite ale Americii. Actualul record mondial la acest concurs a fost de mai puțin de patru secunde în anul 2011.

1.3. Căutare și salvare

De multe ori, din cauza numeroaselor posibilități de accidente care se pot întâmpla, oamenii sunt puși în dificultate în acțiunile de salvare. Din această cauză au fost construite diverse echipamente inteligente care să reușească ușurarea acestor operațiuni efectuate. În funcție de timpul mediului înconjurător, au fost construiți roboți care navighează pe apă precum robotul Emily din figura 1.4, pe uscat, precum și roboți pompieri.



Figura 1.4. Robotul Emily pentru salvarea pe apă [\[4\]](#)

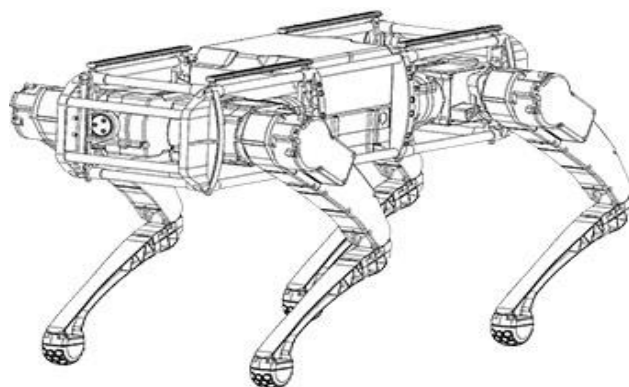


Figura 1.5. Robot pentru cazul unui cutremur [\[5\]](#)

În cazul unui cutremur, roboții pot să se deplaseze pe teren accidentat și pot intra în locuri mai puțin accesibile pentru oameni sau câini. El poate urca sau coborî și unele modele pot căra până la 20 kg de echipament. Sunt construiți cu picioare, cele de dimensiuni mai mari, dar și cu roți sau șenile, cele de dimensiuni reduse. Un astfel de robot este exemplificat în figura 1.5.

Dacă se întâmplă un incendiu, echipamentele sunt programate să caute sursa focului, să o găsească, dar să acționeze în conformitate. Unele din acestea reușesc să deschidă uși sau să le spargă, să transporte o persoană aflată la un loc primejdios, dar să și ajungă la un loc sigur, precum cel din figura 1.6.

Unii din aceștia sunt controlați de la distanță de către oameni, alții sunt programați cum să acționeze în diferite situații. Sunt echipați cu stingătoare care pot fi utilizate la distanțe mari, diferiți senzori pentru a detecta scurgerile de gaze precum și cu viteză ridicată, reușind să traverseze terenuri accidentate.



Figura 1.6. Robot pentru cazuri de incendiu [\[6\]](#)

Un alt tip de echipament de căutare și salvare poate fi și drona. Acestea sunt echipamente cu camere video și tehnologii de recunoaștere a imaginilor pentru a căuta și localiza persoanele dintr-o pădure sau de pe un teren accidentat. Cu ajutorul algoritmilor, drona poate fi ghidată astfel încât să ajungă la persoanele aflate în dificultate.

Acestea sunt doar câteva exemple de roboți de căutare și salvare care pot utiliza algoritmi pentru a găsi ieșirea dintr-un labirint. Datorită tehnologiei și dezvoltării inteligenței artificiale, există, în continuu, noi inovații în acest domeniu.

1.4. Dezamorsare de bombe

Oamenii sunt deseori puși în pericol atunci când vine vorba de a dezamorsa o bombă. Acele bombe amplasate sub diferite forme pot să explodeze în orice moment. Din această cauză ei au ales să trimită roboți programați, reducând astfel riscurile la care erau expuși.



Acești roboți sunt dotați cu multe echipamente de înaltă performanță. Din cauza pericolului foarte mare, nu este bine ca să apară erori sau greșeli. Sunt dotați cu senzori de distanță, senzori infraroșu, senzori de temperatură precum și cu senzori de gaz. Toți aceștia sunt utilizați pentru a detecta obstacole, temperaturi ridicate sau un gaz periculos care pot indica prezența unei situații cu efect explozibil. Cu ajutorul acestora se va construi o hartă pentru a identifica ruta cea mai optimă.

Figura 1.7. Robot pentru dezamorsare bombe [\[7\]](#)

Robotul este echipat cu un braț mult articulată sau alte mecanisme de manipulare pentru a ajunge la explozibil și a o dezamorsa. Controlul de la distanță este utilizat prin intermediul unui sistem de comunicație, astfel încât operatorul să poată monitoriza și ghida acțiunile robotului. Această comunicație transmite informații în timp real despre starea bombelor.

Roboții pot fi de dimensiune mai mică, ușor de transportat, sau mai mare, în funcție de necesitate. Cu toate acestea ei nu au ajuns să înlocuiască o persoană, dar au ajuns să fie de o importanță majoră în domeniul militar.

1.5. Robot aspirator

Pentru a putea ajuta oamenii în ceea ce privește curățenia în casă, au fost construiți roboții aspirator. Aceștia au ca scop de a curăța podeaua și mocheta unei încăperi fără ca să se piardă, să se oprească, activitatea desfășurându-se într-un mod cât se poate de autonom. El poate avea accesorii precum perii rotative și accesorii speciale pentru a curăța eficient în diverse încăperi.

Un astfel de robot aspirator care utilizează algoritmul de deplasare într-un labirint este Roomba, prezentat și în figura 1.8. El este echipat cu senzori de



Figura 1.8. Robot aspirator Roomba [\[8\]](#)

distanță și senzori de coliziune, care permit să detecteze obstacolele dintr-o încăpere. Unele modele mai avansate pot avea și o cameră video sau senzori infraroșu pentru a determina caracteristicile mediului.

Roomba poate utiliza diferite tehnici pentru a-și actualiza poziția continuu în încăpere. Aceasta poate include metode precum odometria (măsurarea distanțelor parcurse de roată) sau folosirea punctelor de referință în încăpere pentru a determina locația relativă a robotului.

Acest robot poate utiliza algoritmi diverși precum cel care este ~~a fost~~ folosit în acest proiect, algoritmi de evitare a obstacolelor, metode de planificare a traseului precum algoritmul de explorare în adâncime sau algoritmul de explorare în înălțime.

Capitolul 2. Componente utilizate

Fiind un sistem complex, acesta a avut nevoie de mai multe componente pentru a putea fi construit. A fost nevoie de următoarele:

- Blocul de comandă care are rol de a programa și controla comportamentul robotului. Pentru aceasta s-a folosit o placă Arduino Nano.
- Un sistem senzorial pentru a detecta informații despre mediul înconjurător. Pentru aceasta au fost folosiți senzori ultrasonici pentru o precizie cât mai bună a datelor.
- Un driver pentru a putea face legătura între sistemul de operare și dispozitivul hardware specifică. Aici s-a folosit un driver Mini DC Motor Dual cu punte H și sistem PWM de comandă.
- Un sistem de deplasare care permite mobilitatea unui obiect, în cazul nostru, fiind robotul și controlul asupra mișcărilor și al vitezei. Sunt folosite motoare și roți aferente pentru aceasta.
- Sistemul de iluminare pentru care este folosit în iluminarea mediului înconjurător și a indica prin semnale luminoase direcția de mers. În aplicație s-au folosit câteva led-uri.
- Sursa de alimentare care furnizează energia necesară pentru funcționarea componentelor și sistemelor robotului. Pentru o ușoară manipulare au fost folosite baterii cu bloc de 9 volți.

Pentru o vizualizare mai clară a componentelor utilizate și modul de comunicare între acestea se poate observa toate elementele în schema-bloc din figura 2.1.

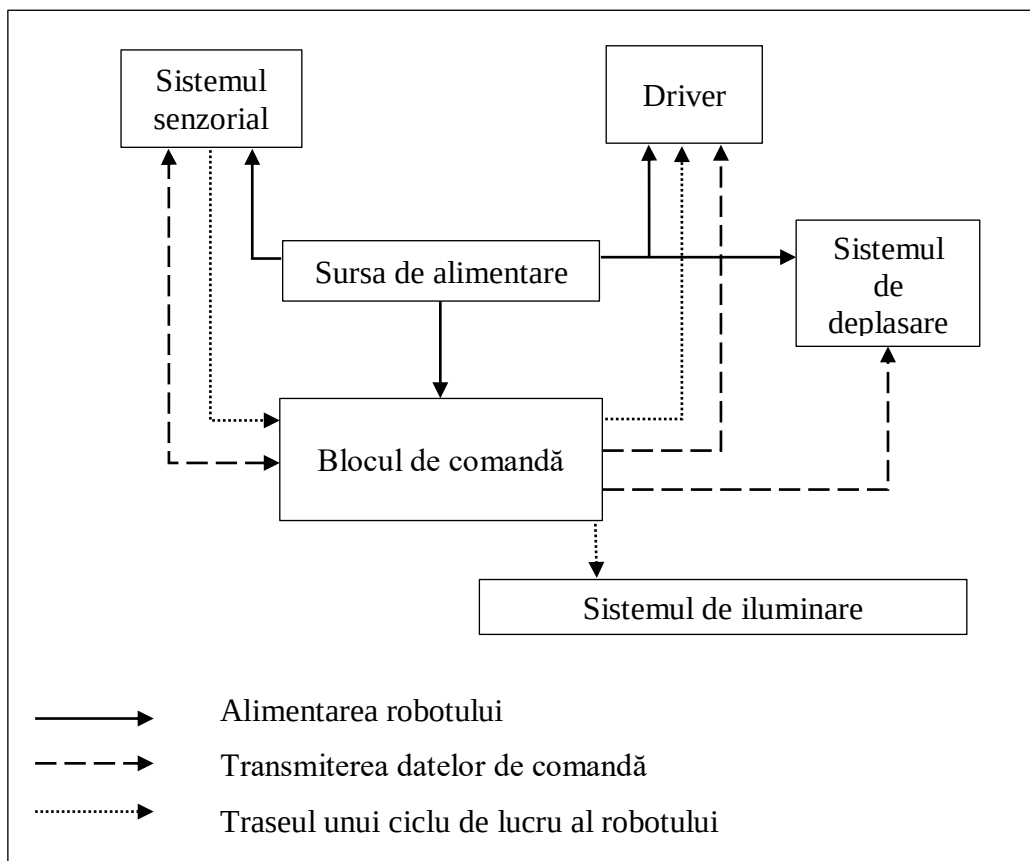


Figura 2.1. Schema bloc pentru robot

Sursa de alimentare este conectă la toate elementele sistemului cu scopul de a menține energia astfel încât sistemul să poată funcționa, acesta fiind în cazul nostru bateriile.

Sistemul senzorial este compus din doi senzori ultrasonici. În urma citirii ciclice, aceștia transferă date către blocul de comandă cu informații legate de distanța până la primul obstacol detectat. Blocul de comandă, la rândul său, transmite datele către driver și sistem de iluminare, în cazul în care există un obstacol aflat la o distanță setată, transmite comenzi driver-ului pentru a face o schimbare de direcție. Driver-ul anunță sistemul de deplasare format din motoare și roți, iar schimbarea de direcție este realizată cu succes.

Sistemul de iluminare este conectat la blocul de comandă, acesta fiind împărțit în două modalități de funcționare diferite. Două leduri ale mașinii sunt aprinse pe tot parcursul traseului executat, pe când celelalte două se aprind semnalizând direcția de deplasare. Dacă acesta va vira spre dreapta sau spre stânga, ledul din dreapta se va aprinde, respectiv cel din stânga.

Comunicația între sistemul senzorial și blocul de comandă este realizată în ambele sensuri, dar către blocul de comandă și sistemul de deplasare sau blocul de comandă și driver, comunicația este doar de la blocul de comandă, nu și invers. Pentru proiectul de față, blocul de comandă este reprezentat de placa Arduino Nano, placă care urmează să fie prezentată mai în detaliu.

2.1. Arduino Nano

Arduino Nano este o placă clasică Arduino concepută cu un microcontroler ATmega328P, care are o arhitectura RISC cu o frecvență de funcționalitate de 16MHz. Acest microcontroler oferă o gamă largă de funcții și capacități, inclusiv un set generos de pini analogici și digitali pentru a conecta senzori, module și alte componente electrice. Sistemele de operare cu care este compatibil acest sistem de dezvoltare sunt Windows, Linux și Mac OS.

ARDUINO NANO

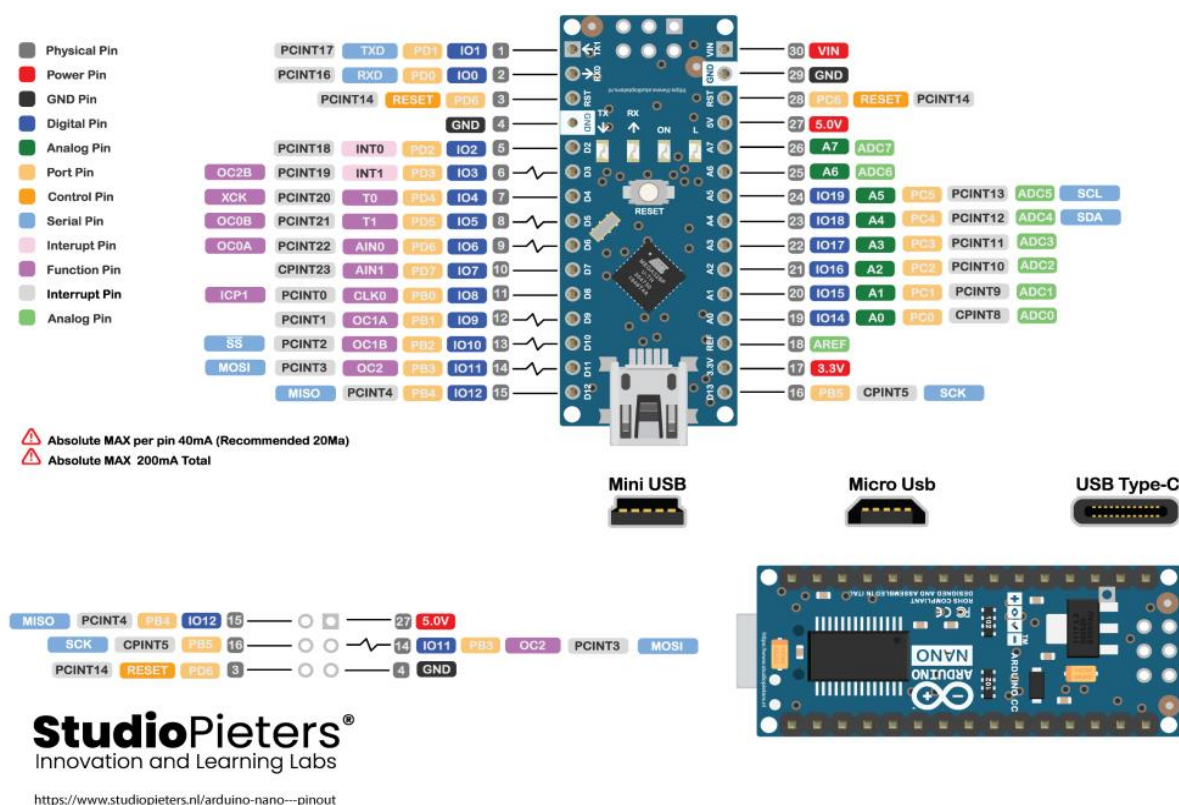


Figura 2.2. Placă Arduino Nano [9]

Placă Arduino Nano păstrează majoritatea funcționalităților și caracteristicilor plăcilor Arduino clasice. Este de mici dimensiuni în comparație cu alte plăci Arduino, versatilă, permițând o atașare ușoară și flexibilă pentru o varietate de proiecte. Cântărește aproximativ 7 grame, dimensiunea fiind de 45x18 mm. Aceasta o face potrivită pentru proiecte în care spațiul este limitat sau pentru aplicații portabile.

Alimentarea acestei plăci, după cum se poate vedea și în figura 2.2, este realizată prin intermediul portului USB sau prin intermediul unui pin de alimentare extern (VIN). Ea suportă o gamă largă de tensiuni de alimentare de la 7 până la 12 de volți. De asemenea, placa dispune de un

regulator de tensiune integrat, care poate converti tensiunea de alimentare la 5V sau 3.3V, în funcție de necesitatea proiectului.

Arduino Nano dispune de un port USB Mini-B pentru conectarea la calculator sau la alte dispozitive. Aceasta permite programarea plăcii și transferul de date între Arduino și alte dispozitive. De asemenea, această placă are pini GPIO (General Purpose Input/Output) pentru conectarea la alte componente sau module, cum ar fi senzori, ecrane, module de monitorizare și multe altele.

Placa Arduino Nano are 32 KB de memorie Flash, din care aproximativ 2 KB din aceștia sunt rezervate pentru bootloader-ul Arduino. De asemenea, dispune de 2 KB de memorie SRAM și 1 KB de memorie EEPROM. Această capacitate de stocare permite implementarea de proiecte complexe și salvarea datelor în timpul execuției.

Specificații tehnice:

- 22 pini intrare/ieșire
- 14 pini digitali de intrare/ieșire (de la D0 la D13)
- Din care 6 pini sunt și PWM (3, 5, 6, 9, 10, 11)
- 8 pini analog de intrare/ieșire (de la A0 la A7) care pot fi folosiți și ca pini digitali
- 2 pini de resetare RST
- Un buton de resetare RST
- GND – pin de referință la masă
- 1 pin configurat ca TX1 pentru comunicație serială
- 1 pin configurat ca RX1 pentru comunicație serială
- 1 pin AREF
- Viteză de ceas de 16MHz
- 3 timer-e ce pot fi configurate în diferite frecvențe și modifică viteza de ceas pe un set de pini la care sunt atașați.
- Suportă comunicații precum SPI, USART, I2C și TWI
- Frecvența de funcționare de 16 MHz

Intrările și ieșirile digitale pot fi utilizate pentru a interacționa cu componente digitale sau pentru a genera semnale digitale. Pini analogici permit citirea valorilor analogice și sunt utilizați pentru a interacționa cu senzori sau alte componente care furnizează date analogice.

Pinii de pe placa de bază, care au fost deja prezentați pot fi clasificați și sub alte forme, semnificând totodată și o metodă de utilizare specifice:

- 2 pini de control (RST)
- 23 pini de întrerupere (PCINT0 – PCINT23)
- Pini funcționali
- 8 pini seriali
- 30 pini fizici

Programarea plăcii Arduino Nano este realizată folosind software-ul Arduino. În partea de unelte ale programului se poate selecta placa, iar comunicația va fi realizată folosind protocolul STK500.

Aceste plăci sunt deseori folosite în diferite domenii. Dintre acestea se numără robotica, sistemele integrate, sistemele de control, educație, automatizare și control, precum și în numeroase produse electrice.

Alte exemple de echipamente care fac parte din familia plăcilor Arduino sunt Arduino Mega, Arduino Pro Mini, Arduino Uno, Arduino Due, Arduino Leonardo, Arduino Lilypad și Arduino YUN, iar alte plăci de dezvoltare care fac parte din această categorie sunt Raspberry Pi, Intel Edison, PIC Development Board, placa ESP32, precum și MSP430 Launchpad.

Concluzionând, Arduino Nano este o soluție compactă pentru dezvoltarea unor numeroase produse electrice. Caracteristicile sale, ușurința de utilizare și gama largă de aplicații afirmă faptul că această placă rămâne una din cele mai populare și accesibile opțiuni pentru diferite proiecte electronice, mai simple sau mai complexe.

2.2. Senzor HC-SR04

Senzorul HC-SR04 se folosește de un sonar pentru a afla distanțele față de anumite obiecte aflate în preajma lui. El poate determina printr-o metodă precisă și non-contact, distanțele. Raza de măsurare este într-un interval de aproximativ 2 cm până la 400 cm, în funcție de setările și configurațiile utilizate.

Precizia este destul de bună în măsurarea distanțelor, având o eroare de aproximativ 3 mm.

Un beneficiu al acestui senzor este faptul că oferă rezultate rapide, având un timp de răspuns de aproximativ 20 de milisecunde.

Senzorii emit ultrasunete de 40.000 Hz cu ajutorul transmițătorului care traversează aerul. Dacă se află un obiect în calea acestor unde, acestea se vor reveni și vor fi primite de receptor. Echivalent efectului Doppler, se poate calcula distanța parcursă dacă se va lua în considerare timpul de la transmiterea semnalului până la receptarea lui, și viteza sunetului.

Privită ca și componentă hardware, aceasta conține un număr de patru pini așezați ca în figura 2.3. Pentru transmițător este folosit - pinul Trig (declanșator), pentru receptor - pinul Echo (ecou), iar pe lângă cele două există pinii VCC (alimentare) și GND (masă) care se conectează la 5V și GND.

Specificații tehnice:

- Tensiunea de alimentare de 5V
- Curent consumat de 15mA
- Unghiul de măsurare de 15 grade
- Eroare de 3mm
- Durata semnalului de intrare de 10μs

Acest senzor nu este afectat de lumina receptată de la soare sau de materiale negre, care ar fi mai greu de detectat la alte tipuri de senzori de același tip. Când este procurat, acesta vine complet echipat cu modul de transmitere și de recepție cu ultrasunete.

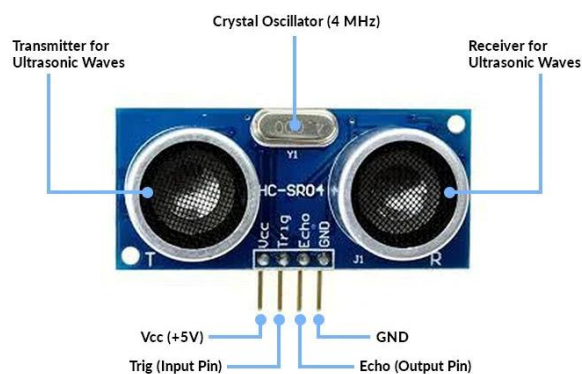


Figura 2.3. Senzor ultrasonic HC-SR04 [\[10\]](#)

În mod practic, pentru a crea o undă de ultrasunet, trebuie să se configureze pinul Trig pe HIGH pentru 10 μ s. Acesta va trimite 8 intervale scurte de unde, care vor călători cu viteza luminii prin aer. Pinii, care receptează undele, vor fi activi abia după ce cele 8 cicluri de transmitere, începând să asculte sau să aștepte fluxul ultrasonic reflectat de obiectul aflat în cale. Dacă nu există un obiect care să reflecte pulsul, pinul Echo se va opri după 38 ms și va fi setat pe LOW. Cu aceste informații, putem calcula distanța cu următoarea formulă:

$$\text{Distanța} = \text{Viteză} \times \text{Timp}$$

După cum putem observa, valorile pentru viteză și timp nu sunt cunoscute. Timpul reprezintă durata în care pinul Echo a rulat ca fiind setat pe HIGH, iar viteza reprezintă viteza luminii care este 340 m/s. Dar avem nevoie de un pas adițional deoarece aceasta valoare reprezintă durata pe care unda sonoră o face până la obiect și înapoi. Deci distanța aflată va trebui să fie împărțită la 2.

Prin urmare, senzorul HC-SR04 poate fi utilizat într-o varietate de aplicații, cum ar fi măsurarea nivelului de apă din rezervoare și monitorizarea distanțelor în sisteme de parcare, și multe altele. Datorită preciziei sale precum și a costului redus, senzorul este unul dintre cei mai cunoscuți pe piață și este deseori folosit în proiecte profesionale.

2.3. Driver

Modulul driver mini DC Motor Dual este deseori folosit pentru automatizare, mașini inteligente, roboți sau alte dispozitive. Cu ajutorul tensiunii de alimentare care variază între 2V și

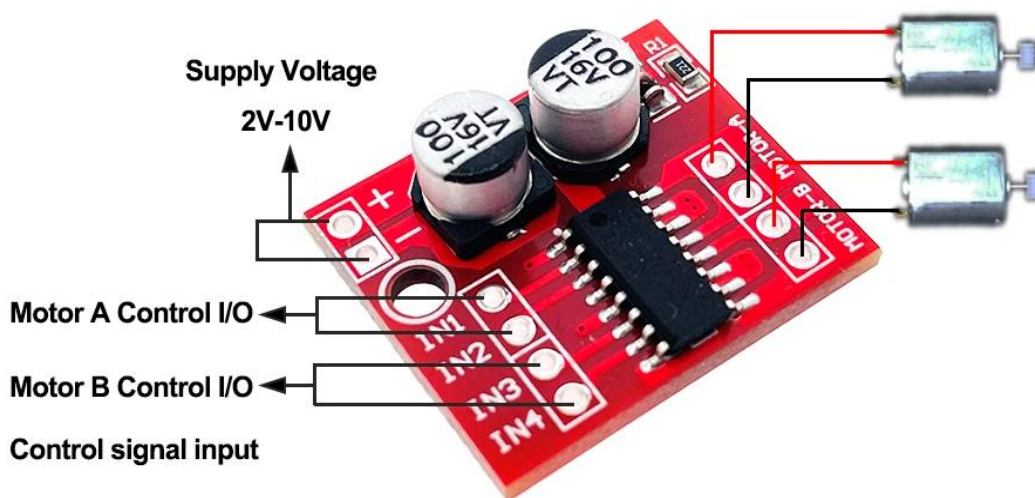


Figura 2.4. Driver Mini DC Motor Dual [\[11\]](#)

10V poate conduce două motoare de curent continuu sau un motor pas cu pas cu 4 fire și 2 faze, prezentat și în figura 2.4. Cu alte cuvinte, acest driver este conceput pentru a facilita controlul bidirecțional al motorului, permițând mișcarea înainte și înapoi.

Driverul are dimensiunea de $24,7 * 21 * 7$ mm și un diametru pentru montare de 2 mm. Ca mod de funcționare, acesta poate realiza o rotație înainte (forward) sau înapoi (reverse), fiind posibilă reglarea vitezei de rotație. Acest control al vitezei de rotație este realizată prin ajustarea tensiunii și curentului furnizat.

Funcționalitatea se bazează pe semnalele trimise de microcontroler. Semnalele primite de driver sunt interpretate prin cei doi pini cu care este configurată placa. Perechea de pini 1 – 2, controlează unul din motoare prin setarea pe unul din pini, sau pe ambele a unei tensiuni de referință și/sau a tensiunii de 0 V. Cealaltă pereche de pini (3 – 4) funcționează similar pentru celălalt motor conectat.

Componentele care sunt folosite pentru crearea unui driver sunt controlerul care poate fi un microprocesor sau un microcontroler, un circuit pentru driver sau un IC care amplifică curentul și convertește semnalele folosite de motoare, 2 condensatoare electrolitice de tensiune înaltă, un stabilizator de tensiune logică.

După cum și menționam, acest driver folosește o punte H. Acest tip de circuit este construit din patru comutatoare care controlează fluxul de curent. În figura 2.5. sarcina este reprezentată de M și conectează două perechi de comutatoare. Folosind o singură sursă de curent, se poate direcționa curentul în două direcții prin închiderea a două comutatoare.

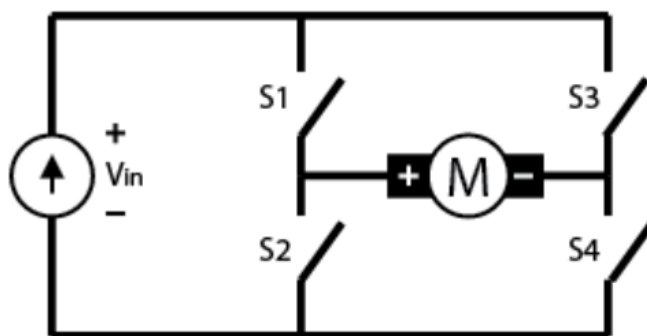


Figura 2.5. Circuit pod H [\[12\]](#)

Dacă comutatoarele 1 și 4 sunt închise, curentul va circula spre dreapta, de la sursă - comutatorul 1, către sarcină - comutatorul 4, apoi înapoi spre sursă. Dacă comutatoarele 2 și 3 sunt închise, curentul va circula spre stânga, de la sursă - comutatorul 3, sarcină - comutatorul 2, apoi înapoi spre sursă. Dacă vor fi închise două comutatoare aflate în serie va cauza un scurtcircuit și va arde podul H.

Driver-ul se bazează pe semnalele modulare în înălțime de impuls sau PWM. Acesta este un semnal digital care variază în timp între două niveluri fixe: nivel înalt (adesea de 5 volți) și nivel scăzut (adesea de 0 volți), prezentat și în figura 2.6.

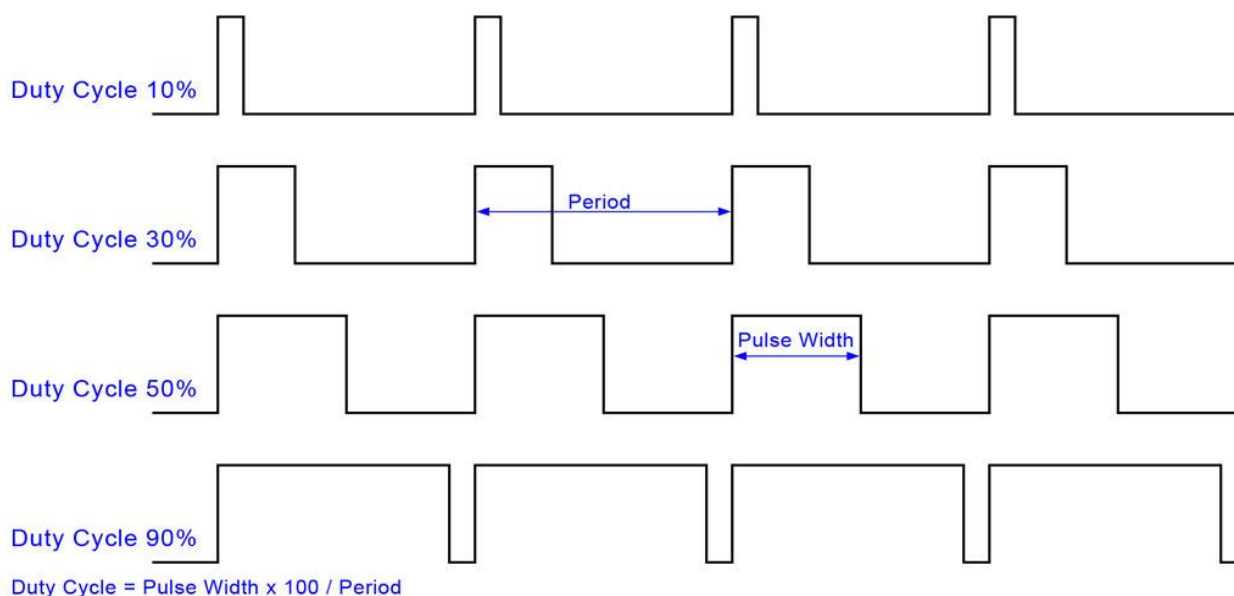


Figura 2.6. Semnal PWM [\[13\]](#)

Principiul de bază al unui semnal PWM constă în modificarea raportului de umplere, adică porțiunile dintre durata în care semnalul este la nivel înalt și durata totală a unui ciclu. Perioada de timp nivelului înalt dintr-un ciclu înalt-scăzut se numește factor de umplere (duty cycle) și reprezintă, în medie, ce tensiune va primi dispozitivul pe care îl folosim. Factorul de umplere este procentual și reprezintă cât la sută din perioada unui semnal, acesta este pe nivelul înalt. Spre exemplu, pentru o tensiune de 5V și un factor de umplere de 10%, semnalul analog rezultat va fi de 0.5V.

Caracteristicile importante ale semnalului PWM sunt amplitudinea (U), frecvența (f) și ciclul de funcționare (S) sau factorul de umplere (D). Cu cât frecvența este mai mare, cu atât este mai mare și precizia de reglare.

Un avantaj major al acestui driver este faptul că este echipat cu protecții împotriva supratensiunii și supracurentului, apărând driverul și motoarele împotriva deteriorării. Există diferite tipuri de drivere care sunt folosite în diferite proiecte. Acestea sunt L293D IC, placa pentru motoare BTS7960B, TB6612FNG, TB6560, PCA9685 16 cu servomotoare, placa MACH3 cu interfața, motor pas cu pas TB6600, placa cu driverul L298.

2.4. Motor de curent continuu

Pentru a aduce la îndeplinire cerințele cerute, am folosit două micro motoare de curent continuu pentru o ușoară deplasare pe parcursul labirintului. Motoarele DC sunt mașinării electrice care transformă energia electrică în energie mecanică.

Motoarele acestea au câteva caracteristici care sunt utilizate pentru a duce la îndeplinire cerințele propuse. Tensiunea de alimentare poate varia între 3V, 6V sau 12V.

Pentru o funcționare corectă și optimă, am ales două motoare care au capacitatea de 6V. Dimensiunea mică de doar 10 x 12 x 26 mm este un avantaj pentru robotul de față.



Figura 2.7. HPCB 250:1 6V Micro Motor [\[14\]](#)

Specificații tehnice:

- Greutate de 9,5g
- Diametrul arborelui de 3 mm
- Raport de transmisie de 250 la 1
- Curent la 6V: 120 mA
- Curent de blocare pentru 6V de 1600 mA
- Cuplu: 0.3kg cm
- Viteza de rulare la 6V: 3000 rpm (rotații pe minut)
- Perii de carbon cu durată lungă de viață

Pentru a putea face anumite mișcări, motorul se folosește de codificator (encoder). Un codificator este un dispozitiv pentru a înregistra mișcarea sau poziția unui obiect. Principalele două tipuri de codificatori sunt codificatorul incremental și codificatorul absolut. Primul tip, cel incremental, furnizează informații despre mișcare în funcție de schimbarea poziției relative față de poziția anterioară. Acesta generează impulsuri de ieșire care sunt contorizate pentru a determina direcția și distanța deplasării. Codificatorul absolut oferă o valoare unică pentru fiecare poziție, astfel încât să poți determina exact obiectul în momentul dat. Acest tip poate utiliza diverse metode pentru a reprezenta valoarea poziției, cum ar fi codul binar sau codul Gray.

Principiul de funcționare a motorului de curent continuu este următorul. În momentul în care un conductor purtător de curent electric este plasat într-un câmp magnetic, se creează o forță mecanică.

Putem observa câteva dezavantaje în această privință. Costurile unui astfel de motor pot fi destul de ridicate. Dar ca avantaj poate fi considerat faptul că putem avea un control al vitezei motorului, timpul de reacție este scurt.

Pentru a funcționa în mod corect, am conectat la aceste două motoare o pereche de roți de dimensiunea 32 x 7 mm, prezentate și în figura 2.8.



Figura 2.8. Roți [\[15\]](#)

2.5. Sisteme periferice

Pentru sursa de alimentare au fost folosite trei baterii de 9V legate în paralel. Bateriile au fost legate în paralel la un conector de baterii. Ele dispun de tehnologia zinc-carbon.

Iluminarea a fost realizată cu ajutorul a patru LED-uri. Termenul de LED semnifică Diodă cu Emisie de Lumină (Light Emitting Diode) și este un dispozitiv semiconductor creat în anul 1927 de Oleg Vladimirovich Losev și comercializat abia în anul 1968. Pentru a putea emite lumină se aplică un voltaj care va ajuta ca joncțiunile P-N să se combine, mărin­d energia și transformând-o sub formă de lumină.



Figura 2.9. Baterie și conector pentru baterie [\[16\]](#)

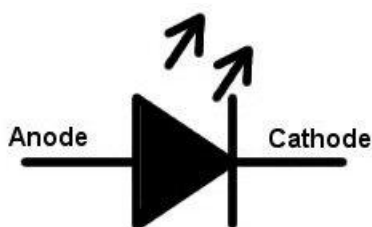


Figura 2.10. Simbolul unui LED [\[17\]](#)

Există mai multe tipuri de LED-uri. Acestea sunt cele de tip lampă sau de tipul unui chip. Simbolul specific pentru el este construit din anod (pozitiv) și catod (negativ), la fel ca în figura 2.10.

Capitolul 3. Considerații de implementare

Folosind placa de dezvoltare Arduino Nano și conectând aceasta la calculator, a fost folosit un mediu integrat de dezvoltare sub numele de Arduino IDE pentru a putea programa toate funcționalitățile.

Pentru început au fost declarate variabilele auxiliare care să scoată în evidență distanțele folosite, numerele de identificare pentru fiecare pin, direcțiile de deplasare a robotului și configurațiile pentru senzorii ultrasonici. Au fost inițializați cei doi pini trig ai senzorilor ultrasonici și cei patru pini pentru motoare de la driver ca pini de ieșire, iar cei doi pini echo ai senzorilor ultrasonici ca pini de intrare. Tot aici sunt inițializați pinii motoarelor și luminile din spate pe LOW (0) pentru a nu fi activi în momentul când programul începe să ruleze.

În bucla principală vor fi executate toate instrucțiunile unde se duce la îndeplinire și tema lucrării de față. Pentru a putea înțelege mai bine modul de funcționare sunt prezentate câteva scheme logice care au fost împărțite în mai multe funcții mai mici, toate explicate în detaliu pe parcurs.

Robotul se bazează pe principiul deplasării într-o matrice ‘*mat*’ unde liniile sunt numerotate cu ‘*x*’ și coloanele sunt numerotate cu ‘*y*’. Valoarea elementului din matrice va fi notată cu 1 sau SET pentru a putea delimita clar traseul pe care îl realizează, restul elementelor fiind 0 sau UNSET. Totodată, pentru o deplasare mai ușoară au fost folosite punctele cardinale nord, vest, sud, est pentru a înainta, a vira la stânga, pentru întoarcere și pentru a vira la dreapta. În cod, pentru a reprezenta acest lucru a fost creată enumerația *direction* cu elementele așezate în ordinea menționată mai sus. La început se setează poziția în care este acum robotul, în matrice cu valoarea 1. Acea poziție este dată de intrarea fizică în labirint, altfel spus, una din pozițiile de la marginile acestuia, urmând ca în funcție de valorile date să se execute algoritmul.

După cum se poate vedea în figura 3.1, ciclul de funcționare al robotului începe cu verificarea dacă se află într-un spațiu deschis sau nu. Condițiile care îi oferă aceste date sunt “*distanceFRONT*” și “*distanceLEFT*” care trebuie să depășească o valoare de control OUTSIDEDISTANCE, o valoare de două ordine mai mare decât distanța dintre robot și un oarecare perete din interiorul labirintului. Cât timp distanțele detectate de senzori nu detectează că a ajuns la finalul labirintului, atunci se citesc din nou distanțele senzorilor și se identifică în ce direcție ar trebui să se deplaseze robotul. Dacă valoarea citită de senzorul din stânga nu detectează un perete în distanța de control setată prin variabila CONTROL_DISTANCE, atunci variabila *step* care indică direcția în care trebuie să se deplaseze mașina în labirint, inițializată cu valoarea de *north*,

va fi incrementată. Altfel, dacă cei doi senzori vor detecta o fundătură, se va considera că robotul are pereți, și în stânga lui, și în fața lui, astfel va vira spre dreapta și variabila *step* va fi decrementată.

Trebuie să luăm în considerare faptul că variabila *step* face parte din enumerația *direction* și în cazul în care ajunge la unul din cele două capete ale acesteia, variabila trebuie setată cu valoarea celuilalt capăt pentru a avea continuitate. Aceasta se realizează cu cele două condiții de

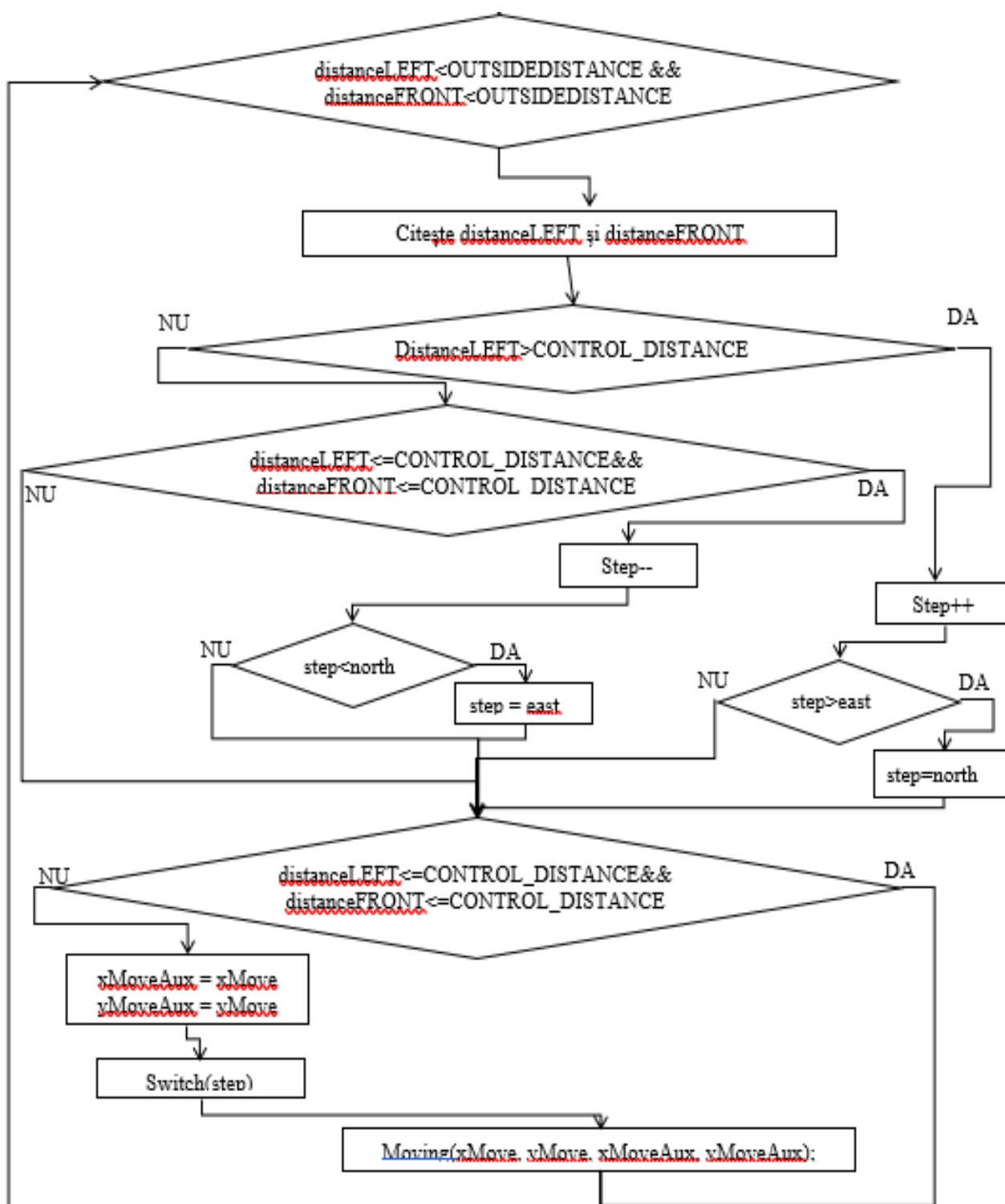


Figura 3.1. Organigrama ciclului principal de funcționare

verificare dacă *step* este mai mare ca *east*, respectiv mai mic decât *north*, variabila *step* să fie egalată cu *north*, respectiv *east*.

După ce variabila *step* este setată conform valorilor citite de cei doi senzori, acum, în funcție de aceasta, se va memora direcția în matrice. Dacă *distanceLEFT* și *distanceFRONT* sunt mai mici sau egale cu distanța de control, atunci nu trebuie să se execute nimic. Altfel procedura începe cu o copie pentru *xMove* și *yMove* semnificând poziția în matrice, se executa un switch în funcție de valoarea *step* și se apelează funcția *moving()*. Valorile *xMove* și *yMove* ne ajută să ne deplasăm în interiorul labirintului care este sub forma unei matrice. Variabila *step* ne spune în ce direcție ar trebui să ne deplasăm în funcție de poziția în care ne aflăm în momentul actual. Toate aceste condiții vor fi executate pentru fiecare pas, inclusiv variabilele distanțelor citite din cei doi senzori, până se ajunge la ieșirea din labirint.

Figura 3.2. prezintă toate cazurile posibile pe care variabila *step* le poate lua. În cazul în care este setat în direcția *north*, *xMove* se va decrementa. Pentru cazul în care este *south*, *xMove* va fi incrementat. Valoarea *yMove* va scade sau va crește dacă *step* este setat cu valoarea *west*, respectiv *east*. Pentru cazul default nu se va executa nici o comandă.

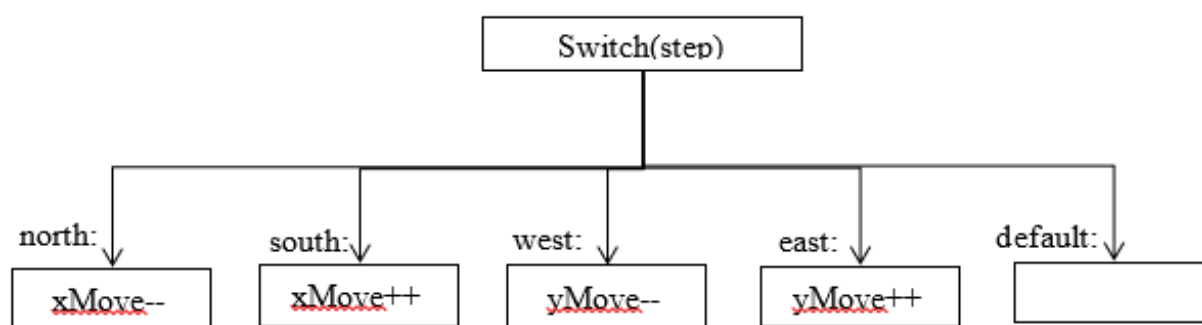


Figura 3.2. Organigrama pentru switch(step)

Funcția *moving()* este apelată cu cele patru valori menționate:

- *xMove* și *yMove* – noile poziții la care robotul trebuie să ajungă
- *xMoveAux* și *yMoveAux* – pozițiile pe care se află robotul în momentul actual

Figura 3.3 începe cu condiția că dacă distanța citită de senzorul ultrasonic din partea stângă este mai mare decât distanța de control, atunci se realizează un viraj spre stânga și se vor face alte verificări (1). Altfel, dacă senzorul din față este mai mare decât distanța de control, robotul se va deplasa un pas înainte și se vor face o serie de alte verificări (2). Dacă nu se respectă nici una din cele două condiții, ambii senzori fiind mai mici decât *CONTROL_DISTANCE*, se va face un viraj spre dreapta și se incrementează variabila *wrong_road*. În cazul în care această variabilă a ajuns să

fie incrementată de două ori consecutiv, aceasta semnifică faptul că robotul a ajuns într-o fundătură și se va semnala aceasta în matrice prin setarea poziției cu valoarea 0.

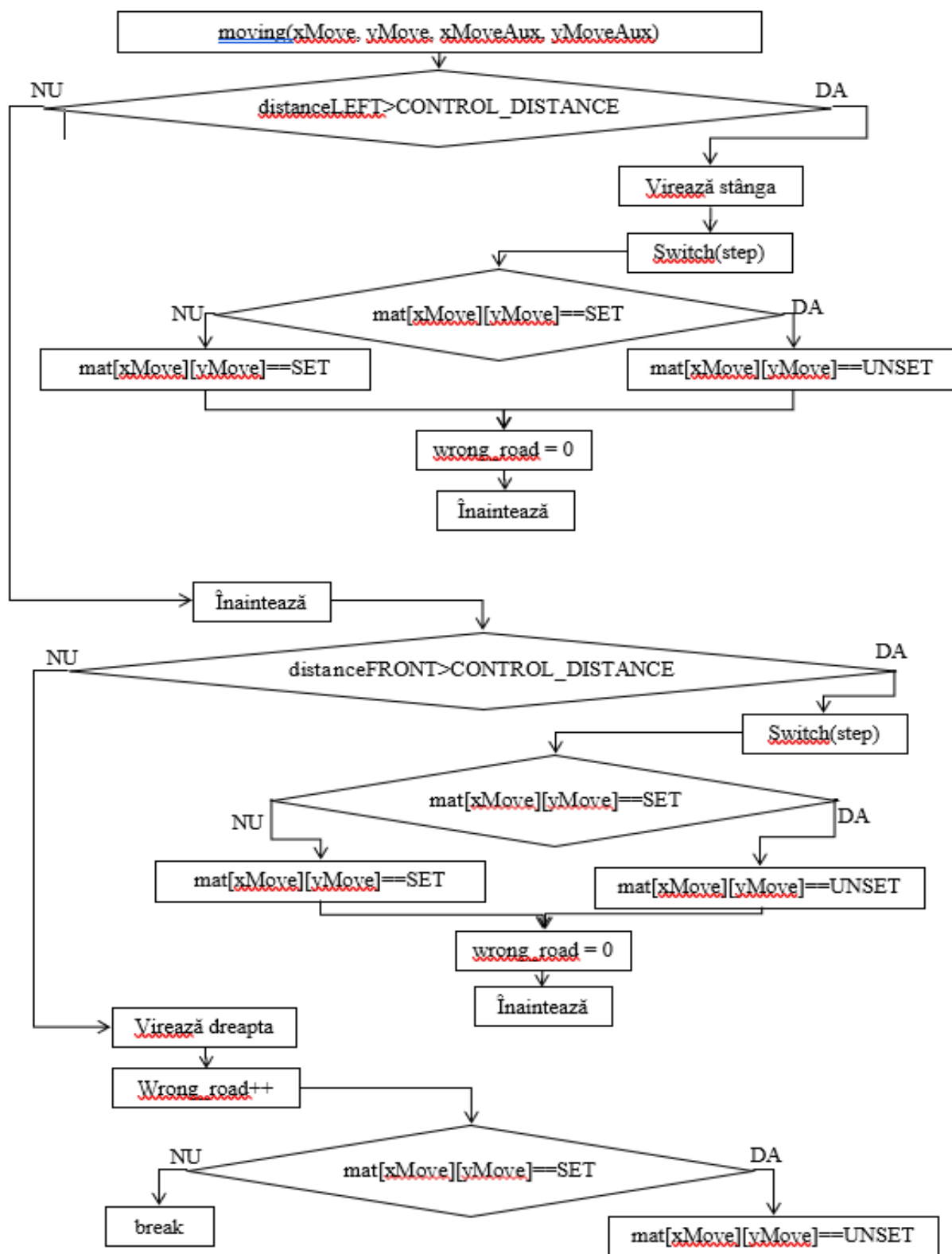


Figura 3.3. Organigrama funcției de deplasare

Pentru prima condiție, după ce robotul virează la stânga (1) este executat un switch care depinde de variabila *step* prezentat și în figura 3.4. Se vor trata toate cazurile de direcție pe care robotul poate să le execute în funcție de poziția curentă, pentru a verifica traseul. Dacă pasul următor trebuie executat spre *north*, se verifică dacă în spatele robotului a fost setată poziția cu

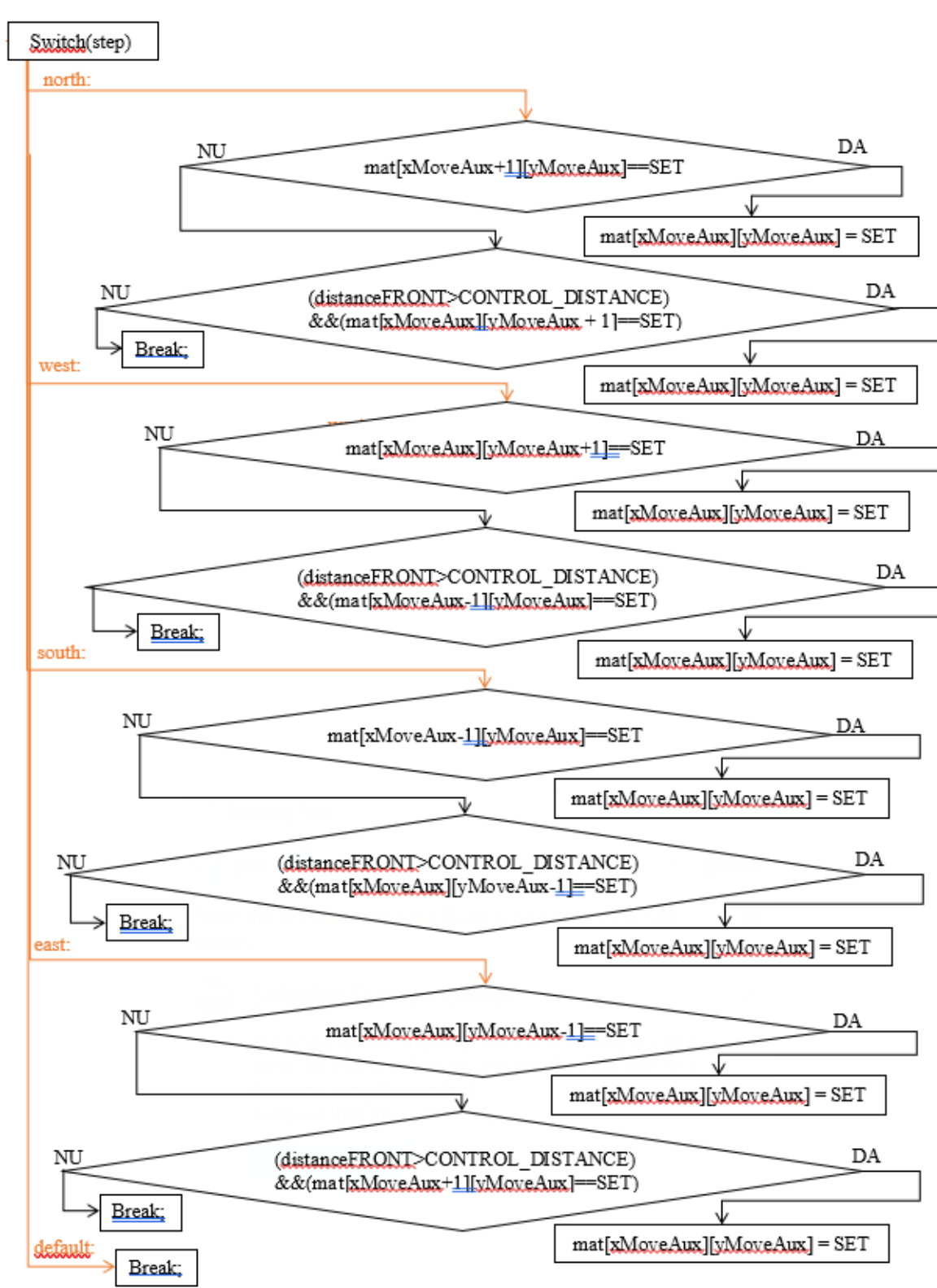


Figura 3.4. Switch pentru cazul în care se deplasează la dreapta

SET adică 1, setăm valoarea curentă cu 1. Altfel se verifică dacă în fața robotului avem unde să ne deplasăm și în dreapta robotului a fost setată poziția pe 1, atunci setăm poziția curentă cu valoarea SET. Se realizează aceeași verificare, dar cu coordonatele specifice.

După care se realizează o verificare adițională pentru cazul în care robotul a parcurs o intersecție. Revenind la figura 3.3. putem observa dacă poziția actuală este setată SET, atunci va fi setată ca UNSET, altfel va fi setată cu valoarea 1. Din moment ce drumul nu a dus la o fundătură se va seta valoarea *wrong_road* cu 0 și robotul va înainta cu un pas, reușind să facă în acest caz doar un viraj, urmând să fie reluat tot procesul de la început pentru alte valori citite.

Pentru a doua condiție, după ce a fost realizat un pas înainte se execută un switch care are ca parametru variabila *step*, prezentat și în figura 3.5.

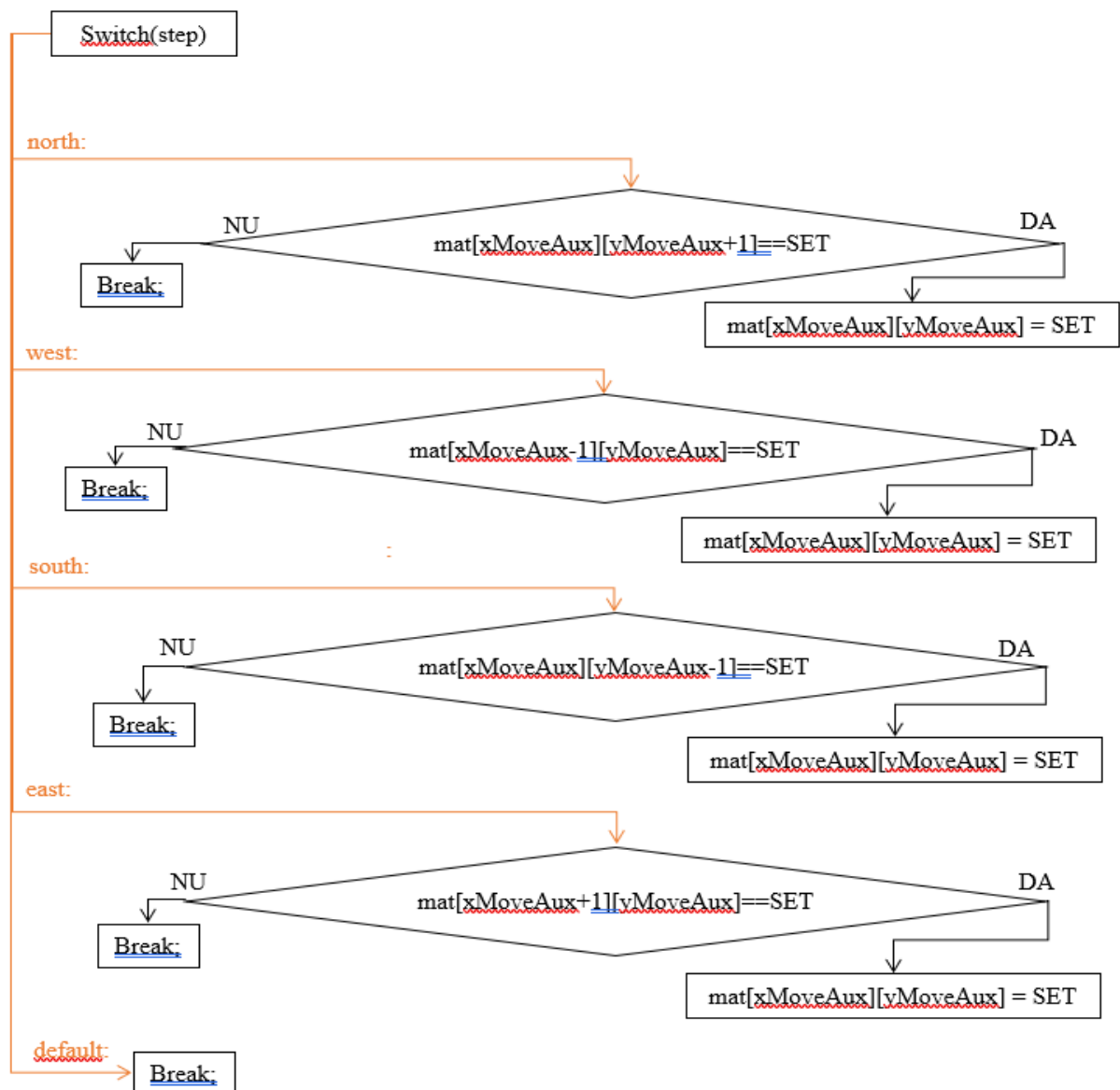


Figura 3.5. Organigrama switch

Și aici sunt parcurse toate cazurile posibile pe care le poate lua variabila step. Pentru cazul în care aceasta este setată pe direcția vest, se verifică dacă poziția anterioară este setată pe 1, atunci valoarea poziției curente va lua valoarea SET. Altfel, nu se va executa nimic, decât o ieșire din acest caz. Se face și o verificare adițională pentru cazul în care a ajuns în intersecție și dacă poziția curentă este setată pe 1, se va seta pe 0, altfel va lua valoarea SET, și fiind un drum corect, wrong_road va lua valoarea 0.

Luminile care au fost conectate la sistem sunt conectate în două moduri. Cele două leduri din fața robotului care semnifică și farurile mașinii, sunt conectate în serie la un pin analogic. Acestea sunt aprinse de la început până la finalul programului. Luminile din spatele mașinii sunt conectate în paralel, la pini digitali diferiți. Acestea sunt puse în funcțiune doar în momentul în care robotul va face un viraj. Dacă virează spre dreapta ledul poziționat în dreapta mașinii se va aprinde, iar dacă virează spre stânga, lumina poziționată în stânga mașinii se va aprinde. Cele două vor fi oprite după ce virajul este executat. Pentru a vedea mai bine această funcționalitate, vom examina codul acestui robot.

Pentru virajul spre dreapta, cum este prezentat și în figura 3.6, a fost realizată o funcție care depinde de cei patru pini ai motoarelor. Doi corespund pentru motorului așezat în partea stângă a robotului, iar doi sunt ai motorului din partea dreaptă. Pentru a putea face virajul spre dreapta motorul din dreapta trebuie să se oprească și cel din stânga să se deplaseze înainte. Pentru aceasta sunt setate corespunzător pe HIGH și LOW. După ce virajul este executat, ledul analog din partea dreaptă este aprins prin setarea acestuia cu valoarea 500. De asemenea, este adăugată și o întârziere.

```
void turnRight(int motorLeft1, int motorLeft2, int motorRight1, int motorRight2)
{
    digitalWrite(motorLeft1, HIGH);
    digitalWrite(motorLeft2, LOW);
    digitalWrite(motorRight1, HIGH);
    digitalWrite(motorRight2, HIGH);
    analogWrite(ledBackR, 500);
    delay(700);

    stopMotors(motorLeft1, motorLeft2, motorRight1, motorRight2);
}
```

Figura 3.6. Cod viraj dreapta

Virajul spre stânga are aceeași structură ca cel care se îndreaptă spre dreapta. În figura 3.7. vedem cum de data aceasta singurul motor oprit este cel din partea stângă.

```

void turnLeft(int motorLeft1, int motorLeft2, int motorRight1, int motorRight2)
{
    digitalWrite(motorLeft1, HIGH);
    digitalWrite(motorLeft2, HIGH);
    digitalWrite(motorRight1, HIGH);
    digitalWrite(motorRight2, LOW);
    analogWrite(ledBackL, 500);
    delay(700);
}

```

Figura 3.7. Cod viraj stânga

Pentru deplasarea înainte prin labirint, a fost creată o funcție separată, folosind aceleași componente, dar având condiții diferite. Figura 3.8. demonstrează faptul că ambele motoare au câte un pin setat pe LOW și un pin setat pe HIGH.

```

void goForword(int motorLeft1, int motorLeft2, int motorRight1, int motorRight2)
{
    digitalWrite(motorLeft1, HIGH);
    digitalWrite(motorLeft2, LOW);
    digitalWrite(motorRight1, HIGH);
    digitalWrite(motorRight2, LOW);
    delay(700);
    stopMotors(motorLeft1, motorLeft2, motorRight1, motorRight2);
}

```

Figura 3.8. Cod pentru înaintarea robotului cu un pas

Pentru ca robotul să se oprească trebuie ca toți cei patru pini să fie setați pe HIGH, ca în figura 3.9. Astfel motoarele din partea dreaptă și din partea stângă își vor opri funcționalitatea.

```

void stopMotors(int motorLeft1, int motorLeft2, int motorRight1, int motorRight2)
{
    digitalWrite(motorLeft1, HIGH);
    digitalWrite(motorLeft2, HIGH);
    digitalWrite(motorRight1, HIGH);
    digitalWrite(motorRight2, HIGH);
}

```

Figura 3.9. Cod pentru oprirea celor două motoare

O altă funcție importantă care a fost creată și utilizată foarte mult este cea de citire a distanțelor cu ajutorul senzorilor ultrasonici. Bazat pe principiul de funcționare prezentat în capitolul 2, a fost creat codul din figura 3.10. Este declarată o variabilă de tipul long care va memora durata în care semnalul de la modul, este pe HIGH, și este transmis sub formă de pulsuri timp de câteva microsecunde, returnând în final valoarea distanței parcurse.

```

int readSensorDist(int echoPin, int trigPin)
{
    long duration;
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(2);

    digitalWrite(trigPin, HIGH);
    delayMicroseconds(2);
    digitalWrite(trigPin, LOW);

    duration = pulseIn(echoPin, HIGH);
    return (duration * 0.034 / 2); //return distance
}

```

Figura 3.10. Cod pentru citirea distanțelor cu ajutorul senzorilor ultrasonici

Capitolul 4. Rezultate experimentale

Carcasa pentru robot a fost construită, cu ajutorul imprimantei 3D, o carcasă cu rolul de a proteja componentele interne sensibile, cum ar fi placa Arduino Nano, senzorii - împotriva șocurilor, prafului, umidității sau ale altor factori de mediu care ar putea afecta funcționalitatea corectă a robotului. Ea mai servește ca și structură de bază pentru asamblarea componentelor robotului, oferind suport și stabilitate și, totodată, asigură aspectul estetic.

Carcasa a fost realizată în CATIA (Computer Three-Dimensional Interface Application) o aplicație dezvoltată de compania Dassault Systèmes. Este una dintre cele mai puternice și versatile aplicații de proiectare pe calculator disponibile pe piață, fiind folosită de ingineri, arhitecți sau designeri pentru a crea și inova sisteme complexe și modele 3D în industrii precum cea aerospațială, automotive, industria civilă și multe altele.

Aplicația oferă o gamă largă de instrumente pentru analiză și simulare avansată într-un mediu virtual. Utilizatorii pot evalua performanța produsului în diferite scenarii, pot efectua analize de rezistență, analize termice, analize de flux și multe altele. Utilizatorii pot crea geometrie precisă și pot defini relații între diferitele componente ale modelului. Putem observa rezultatul final și precizia acestei aplicații prin carcasa realizată, dar și din figura 4.1.

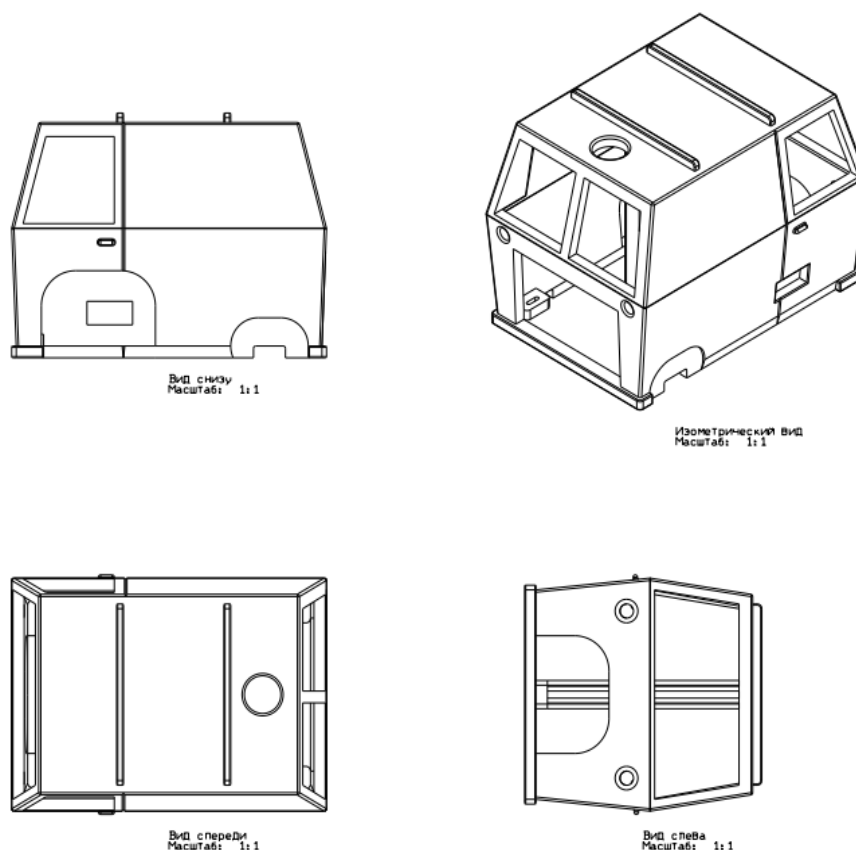


Figura 4.1. Carcasă proiectată în CATIA

Pentru a putea pune în funcțiune sistemul, acesta trebuie alimentat cu 2 baterii de 9V legate în paralel. După ce acestea au fost conectate, robotul este gata de utilizare. Pe acoperișul mașinii se observă butonul de alimentare a circuitului (figura 4.2.) care se setează pe 1, pentru a pune totul în funcțiune.



Figura 4.3. Robot din față

Semnalele luminoase din fața robotului (figura 4.3.) se vor aprinde, fiind în această stare pe

tot parcursul traseului. Senzorii ultrasonici, poziționați în față și pe partea stângă a robotului (figura 4.4.), vor începe să detecteze distanțele până la obiectul aflat în raza lor de detecție. Robotul va parcurge algoritmul dezvoltat pe

microcontroler, aducând la îndeplinire scopul pentru care a fost conceput.

Pe partea dreaptă a robotului (figura 4.5) a fost conceput un loc pentru a putea conecta ușor placa Arduino Nano la calculator, testarea realizându-se într-un timp optim.



Figura 4.4. Robot laterala stângă



Figura 4.5. Robot laterala dreaptă

În partea din spate există un spațiu special pentru a putea avea acces ușor la baterii și la celelalte componente. Un avantaj al carcasei utilizate este și faptul că este detașabilă. Ea poate fi îndepărtată, având acces deplin la toate componentele robotului.

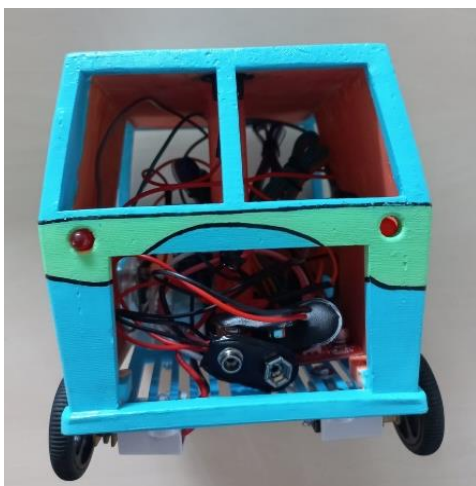


Figura 4.6. Robot din spate



Figura 4.7. Robot de jos

Pentru a testa efectiv comportamentul și modul de funcționare a acestui sistem robotizat a fost creat un labirint. Lățimea totală a acestuia este de 120 metrii și lungimea este de 120m. Distanța în care robotul trebuie să meargă este de aproximativ 30 de centimetrii, perfectă pentru a putea lua orice viraj. Labirintul este cel din figura 4.8.



Figura 4.8. Labirint

Capitolul 5. Concluzii și direcții viitoare de dezvoltare

Proiectul de licență s-a centralizat pe dezvoltarea și implementarea unui robot capabil să navigheze și să găsească drumul prin labirint. Prin intermediul senzorilor robotul este capabil să identifice și să evite obstacolele din labirint și să ajungă la ieșire. El memorează traseul cel mai scurt care ar putea fi parcurs din prima parcurgere realizată. Algoritmul a fost creat și testat special pentru cazul acestui proiect.

Robotul poate fi îmbunătățit prin parcurgerea matricei deja memorate prin algoritm. Aceasta poate fi realizată prin diferite metode precum: crearea unui buton adițional care să delimiteze codul de învățare și partea de cod care parcurge labirintul deja învățat, adăugarea unei pauze mai lungi în care robotul este mutat în poziția de start parcurgând traseul învățat sau prin traversarea labirintului de la final la început.

O altă îmbunătățire poate fi partea de cod și ca robotul să poată parcurge anumite intersecții pe diagonală. Aceasta ar optimiza durata de învățare. Se poate adăuga și anumiți senzori cu o melodie pentru momentul în care robotul parcurge labirintul, pentru a putea ține publicul în suspans.

Concluzionând, acest proiect a oferit oportunitatea de a explora și de a aplica cunoștințele teoretice și practice acumulate în domeniul robotică, algoritmică și inteligență artificială. El a demonstrat și prin această metodă potențialul și utilitatea roboților autonomi în trasee complexe și domenii diferite de dezvoltare și utilitate.

Capitolul 6. Bibliografie

1. https://s13emagst.akamaized.net/products/20689/20688881/images/res_fa9d156cefd521ee0dfd14d8886cfdcf.jpg
2. <https://jerrywu.dev/images/micromouse-maze.jpg>
3. <https://assets.rbl.ms/25566060/origin.jpg>
4. <https://www.edarley.com/media/catalog/product/cache/0b0f9262830dfdc0461ad9dc01e7260b/2/-/2-views-sonar-ko.jpg>
5. https://www.asme.org/getmedia/129AE960-F8CC-489C-B6CA-BB3D47B29A6C/Robots-to-the-Rescue_01.aspx?data-width-removed=340
6. <https://sc04.alicdn.com/kf/H1ae1e5f038f2479891ff49b179480db32.jpg>
7. https://media.defenseromania.ro/image/201911/full/common_robotic_system_prc-e2prc-80prc-93_heavy_12991800.jpg
8. https://www.irobot.ro/app/uploads/2019/10/i7_12x-482x0-c-default.jpg
9. https://www.google.com/url?sa=i&url=https%3A%2F%2Fwww.studiopieters.nl%2Farduino-pinout%2F&psig=AOvVaw0Qgd1FAY1jvF1YlssnZYxA&ust=1684588398319000&source=images&cd=vfe&ved=0CBEQjRxqFwoTCPCsj_y6gf8CFQAAAAAdAAAAABAH
10. <https://mybots.in/wp-content/uploads/2022/07/mybots-88125-02.webp>
11. https://ae01.alicdn.com/kf/H2438b560cfbb48a79639b66e0837ce1bm/1PCS-2-Way-DC-Motor-Drive-Module-Reversing-PWM-Speed-Dual-H-Bridge-Stepper-Mini-Victory.jpg_Q90.jpg_.webp
12. <https://digilent.com/blog/wp-content/uploads/2014/12/H-bridge.png>
13. <https://www.fpvromania.ro/blogimg/1472663144-Semnal.jpg>
14. https://static.optimusdigital.ro/27153-thickbox_default/hpcb-pololu-2501-6-v-micro-motor.jpg
15. <https://a.pololu-files.com/picture/0J10440.1200.jpg?44318cdc8b5116945880482c26aa817c>
16. https://www.videoprint.ro/143832-storage_uploads/50pcs-lot-baterie-9v-conector-snap-clip-adaptor-de-1.jpeg
17. https://th.bing.com/th/id/R.866589f2e7abbe15df5d76c2fa26771f?rik=qQJT32j00Qob4w&riu=http%3a%2f%2f4.bp.blogspot.com%2f-C2xougihqAw%2fvC4NsdLBFBFI%2fAAAAAAAABFY%2fglEm7rQ6xqg%2fs1600%2fled_simbol2

[.jpg&ehk=WHCjcqYapzMR3tCmHwgYNGf7%2f5h2McDpU%2fg5BAP7oEQ%3d&risl=&pid=Im
gRaw&r=0](#)

18. <https://www.colorful.hr/robotizarea-a-patra-revolutie-industrial-a-ne-bate-la-usa-ce-e-de-facut/>
19. <https://www.youtube.com/watch?v=ZMQbHMgK2rw>
20. [History - Micromouse Online](#)
21. https://attend.ieee.org/r2sac-2020/wp-content/uploads/sites/175/2020/01/MicroMouse_Rules_2020.pdf
22. https://lcdn.altex.ro/resize/media/catalog/product/A/S/2bd48d28d1c32adea0e55139a4e6434a/ASPMIMOPPRO2PLU_3.jpg
23. <https://www.elprocus.com/an-overview-of-arduino-nano-board/>
24. https://carpentries-incubator.github.io/IoT_arduino_nano/02-Arduino-Nano/index.html
25. <https://roboindia.com/tutorials/arduino-nano-general-introduction/>
26. https://howtomechatronics.com/tutorials/arduino/ultrasonic-sensor-hc-sr04/?utm_content=cmp-true
27. <https://www.instructables.com/Tutorial-for-Dual-Channel-DC-Motor-Driver-Board-PW/>
28. <https://robocraze.com/blogs/post/what-is-motor-driver>
29. <https://digilent.com/blog/what-is-an-h-bridge/>
30. <https://www.modularcircuits.com/blog/articles/h-bridge-secrets/h-bridges-the-basics/>
31. <https://www.fpvromania.ro/blog/pwm-9.html>
32. <https://know.electricianexp.com/ro/knowledgebase/chto-takoe-shim>
33. <https://www.iqsdirectory.com/articles/electric-motor/dc-motors.html>
34. <https://www.pololu.com/product/3068>
35. <https://www.pololu.com/product/1087>
36. <https://www.rohm.com/electronics-basics/leds/what-are-leds>

Anexe

Codul sursă

```
/****** defines *****/
#define CONTROL_DISTANCE 15 //cm
#define CONSECUTIVE_RIGHT_ROTATION 2
#define OUTSIDEDISTANCE 1000
#define MATRIXLEN 20
#define echoPinLeft 10
#define trigPinLeft 9
#define echoPinFront 12
#define trigPinFront 11
#define SET 1
#define UNSET 0
#define ledFront A3
//#define ledFrontL 2
#define ledBackR A1
#define ledBackL A2
/****** variables *****/
int distanceLEFT=100, distanceFRONT=100;
bool mat[MATRIXLEN][MATRIXLEN];
enum direction {north = 1, west, south, east};
int xMove = 19, yMove = 9, wrong_road = 0, Step = north;
int xMoveAux = 0, yMoveAux = 0;
//motor A connections
int motorLeft1 = 6;    //inF1
int motorLeft2 = 7;    //inF2
// Motor B connections
int motorRight1 = 4;   //inL1
int motorRight2 = 5;   //inL2

/****** prototype *****/
void moveing(int xMove, int yMove, int xMoveAux, int yMoveAux);
void showMatrix(bool a[MATRIXLEN][MATRIXLEN]);
int readSensorDist(int echoPin, int trigPin);
void printDistance(int distance, char nameSensor);
void turnLeft(int motorLeft1, int motorLeft2, int motorRight1, int motorRight2);
void turnRight(int motorLeft1, int motorLeft2, int motorRight1, int motorRight2);
void goForword(int motorLeft1, int motorLeft2, int motorRight1, int motorRight2);
void stopMotors(int motorLeft1, int motorLeft2, int motorRight1, int motorRight2);
void stopGoing(int motorLeft1, int motorLeft2, int motorRight1, int motorRight2);
void goBackword(int motorLeft1, int motorLeft2, int motorRight1, int motorRight2);
```

```

/***** setup *****/
void setup()
{
  /* Set all the motor control pins to outputs*/
  Serial.begin(9600);
  pinMode(motorLeft1, OUTPUT);
  pinMode(motorLeft2, OUTPUT);
  pinMode(motorRight1, OUTPUT);
  pinMode(motorRight2, OUTPUT);

  pinMode(echoPinLeft, INPUT);
  pinMode(trigPinLeft, OUTPUT);

  pinMode(echoPinFront, INPUT);
  pinMode(trigPinFront, OUTPUT);

  /* Turn off motors - Initial state
    LOW = 0
    HIGH = 1
  */
  digitalWrite(motorLeft1, LOW);
  digitalWrite(motorLeft2, LOW);
  digitalWrite(motorRight1, LOW);
  digitalWrite(motorRight2, LOW);

  /* The front LEDs are connected in series and will
    be switched on from the beginning to the end of the route*/
  analogWrite(ledFront, 500);

  /* Leds are set on off because they are used when a turn is made*/
  analogWrite(ledBackR, LOW);
  analogWrite(ledBackL, LOW);
}

```

```

/***** loop *****/
void loop()
{
    /* Set l in matrix in the poz that is the robot */
    mat[xMove][yMove] = SET;

    //verify if you are still in the maze
    while (distanceLEFT < OUTSIDEDISTANCE && distanceFRONT < OUTSIDEDISTANCE)
    {
        /* The two ultrasonic sensors on the front and on the left detect
        the new information about the route */
        distanceLEFT = readSensorDist(echoPinLeft, trigPinLeft);
        distanceFRONT = readSensorDist(echoPinFront, trigPinFront);

        /* Verify that you can go in left and modify 'Step' value */
        if(distanceLEFT > CONTROL_DISTANCE)    //distLeft>50
        {
            Step++;
            /* If Step reaches the last value in the enum, the first value is set */
            if(Step>east)
            {
                Step=north;
            }
        }
        else
        {
            /* Verity that bouth of the sensors detects wall and modify 'Step' value */
            if(distanceLEFT <= CONTROL_DISTANCE && distanceFRONT <= CONTROL_DISTANCE)
            {
                Step--;
                /* If Step reaches the first value in the enum, the last value is set */
                if(Step<north)
                {
                    Step = east;
                }
            }
        }
    }
}

```

```

/* Verity that bouth of the sensors detects wall and do nothing */
if((distanceLEFT <= CONTROL_DISTANCE) && (distanceFRONT <= CONTROL_DISTANCE))
{
    /* Do nothing */
}
else
{
    /* Create beckup for actual position of the robot.
    - xMoveAux and yMoveAux are the values of the position that the robot has now
    - xMove and yMove are the values of a position that the robot will take in the next step
    */
    xMoveAux = xMove;
    yMoveAux = yMove;

    /* Set x and y positions will take in mat[][] depending on the next 'Step' */
    switch(Step)
    {
        case north:
            xMove--;
            break;
        case south:
            xMove++;
            break;
        case west:
            yMove--;
            break;
        case east:
            yMove++;
            break;
        default:
            break;
    }
}

/* The robot is moveing depending on current and future x and y positions */
moving(xMove, yMove, xMoveAux, yMoveAux);

/* The motors are stoped and is waiting for the following values detected by the sensors */
stopMotors(motorLeft1,motorLeft2,motorRight1,motorRight2);

/* The two LEDs behind the robot are off */
analogWrite(ledBackL,LOW);
analogWrite(ledBackR,LOW);

/* A delay is added to the code for the robot. It should go step by step, not run continuously */
delay(700);
}
}

/***** functions *****/
void moving(int xMove, int yMove, int xMoveAux, int yMoveAux)
{
    /* Conditions if you can go LEFT */
    if (distanceLEFT > CONTROL_DISTANCE)
    {
        Serial.println("Turn LEFT");
        //index = 0;
        turnLeft(motorLeft1, motorLeft2, motorRight1, motorRight2);    //turn LEFT
    }
}

```



```

/* Depending on the Step, the robot take next turn */
switch (Step)
{
    case north:
        if (mat[xMoveAux + 1][yMoveAux] == SET)
        {
            mat[xMoveAux][yMoveAux] = SET;
        }
        else
        {
            if ((distanceFRONT > CONTROL_DISTANCE) && (mat[xMoveAux][yMoveAux + 1] == SET))
            {
                mat[xMoveAux][yMoveAux] = SET;
            }
        }
        break;

    case west:
        if (mat[xMoveAux][yMoveAux + 1] == SET)
        {
            mat[xMoveAux][yMoveAux] = SET;
        }
        else
        {
            if ((distanceFRONT > CONTROL_DISTANCE) && (mat[xMoveAux - 1][yMoveAux] == SET))
            {
                mat[xMoveAux][yMoveAux] = SET;
            }
        }
        break;

    case south:
        if (mat[xMoveAux - 1][yMoveAux] == SET)
        {
            mat[xMoveAux][yMoveAux] = SET;
        }
        else
        {
            if ((distanceFRONT > CONTROL_DISTANCE) && (mat[xMoveAux][yMoveAux - 1] == SET))
            {
                mat[xMoveAux][yMoveAux] = SET;
            }
        }
        break;

    case east:
        if (mat[xMoveAux][yMoveAux - 1] == SET)
        {
            mat[xMoveAux][yMoveAux] = SET;
        }
        else
        {
            if ((distanceFRONT > CONTROL_DISTANCE) && (mat[xMoveAux + 1][yMoveAux] == SET))
            {
                mat[xMoveAux][yMoveAux] = SET;
            }
        }
        break;

    default:
        break;
}
//end switch(Step)

```

```

    if (mat[xMove][yMove] == SET)
    {
        mat[xMove][yMove] = UNSET;
    }
    else
    {
        mat[xMove][yMove] = SET;
    }

    /* The robot could turn right or left, so it is not a wrong road */
    wrong_road = 0;

    /* Afthe that the robot turn left, he goes one step forward becaouse there is free*/
    goForward(motorLeft1, motorLeft2, motorRight1, motorRight2);

}
else
{
    /* Conditions if you can go FORWARD */
    if (distanceFRONT > CONTROL_DISTANCE)
    {
        //Serial.println("go FORWARD");
        /* The robot goes foeword one step beause is a free place */
        goForward(motorLeft1, motorLeft2, motorRight1, motorRight2);

        /* Depending on the Step, the robot take next turn */
        switch (Step)
        {
            case north:
                if (mat[xMoveAux][yMoveAux + 1] == SET)
                {
                    mat[xMoveAux][yMoveAux] = SET;
                }
                break;

            case west:
                if (mat[xMoveAux - 1][yMoveAux] == SET)
                {
                    mat[xMoveAux][yMoveAux] = SET;
                }
                break;

            case south:
                if (mat[xMoveAux][yMoveAux - 1] == SET)
                {
                    mat[xMoveAux][yMoveAux] = SET;
                }
                break;

            case east:
                if (mat[xMoveAux + 1][yMoveAux] == SET)
                {
                    mat[xMoveAux][yMoveAux] = SET;
                }
                break;
            default:
                break;
        }
    }
}

```

```

        if (mat[xMove][yMove] == SET)
        {
            mat[xMove][yMove] = UNSET;
        }
        else
        {
            mat[xMove][yMove] = SET;
        }
    /* The robot could turn right or left, so it is not a wrong road */
    wrong_road = 0;

}

/* Conditions if you can go RIGHT and bouth and the two sensors on the right and front detect wall */
else
{
    /* Turn Right because there is a way in the maze */
    turnRight(motorLeft1, motorLeft2, motorRight1, motorRight2);

    /* If turns right it might be a wrong road */
    wrong_road++;

    /* If turns two consecutive times in right, means is a worg road. */
    if (wrong_road == CONSECUTIVE_RIGHT_ROTATION)
    {
        /* Reset the position in the matrix */
        mat[xMove][yMove] = UNSET;
    }
}
}

int readSensorDist(int echoPin, int trigPin)
{
    long duration;
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(2);

    digitalWrite(trigPin, HIGH);
    delayMicroseconds(2);
    digitalWrite(trigPin, LOW);

    duration = pulseIn(echoPin, HIGH);
    return (duration * 0.034 / 2); //return distance
}

void turnLeft(int motorLeft1, int motorLeft2, int motorRight1, int motorRight2)
{
    digitalWrite(motorLeft1, HIGH);
    digitalWrite(motorLeft2, HIGH);
    digitalWrite(motorRight1, HIGH);
    digitalWrite(motorRight2, LOW);
    analogWrite(ledBackL, 500);
    delay(700);

    stopMotors(motorLeft1, motorLeft2, motorRight1, motorRight2);
}

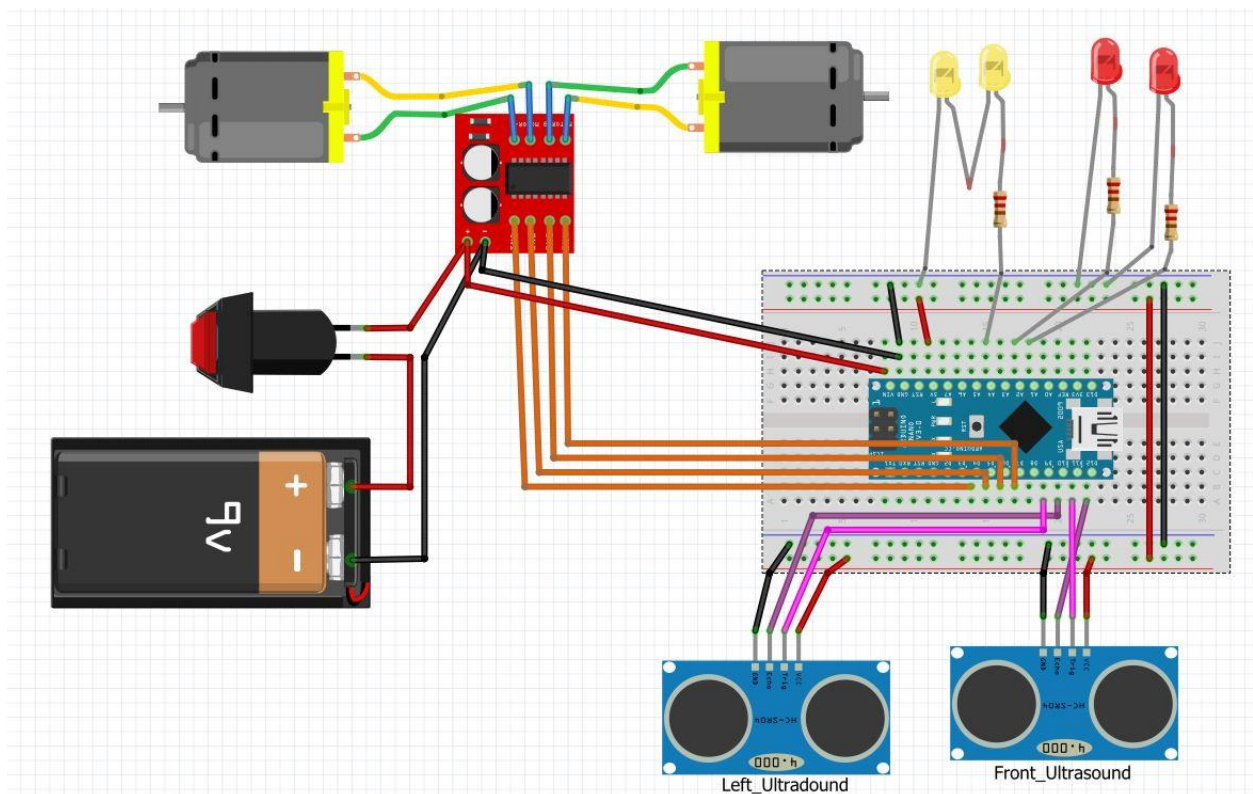
```

```

void turnRight(int motorLeft1, int motorLeft2, int motorRight1, int motorRight2)
{
    digitalWrite(motorLeft1, HIGH);
    digitalWrite(motorLeft2, LOW);
    digitalWrite(motorRight1, HIGH);
    digitalWrite(motorRight2, HIGH);
    analogWrite(ledBackR, 500);
    delay(700);

    stopMotors(motorLeft1, motorLeft2, motorRight1, motorRight2);
}
void goForward(int motorLeft1, int motorLeft2, int motorRight1, int motorRight2)
{
    digitalWrite(motorLeft1, HIGH);
    digitalWrite(motorLeft2, LOW);
    digitalWrite(motorRight1, HIGH);
    digitalWrite(motorRight2, LOW);
    delay(700);
    stopMotors(motorLeft1, motorLeft2, motorRight1, motorRight2);
}
}

```



Schema electrică a circuitului realizată prin aplicația Fritzing.